

# Building Generative UI with MCP in React

Glenn Reyes

React Alicante · September 24, 2026

# The Conference Agenda Assistant

One tool. Twelve fictional sessions.

A list, a comparison and a filter.

 Your conference, your pace

Sample agenda

## Find your next session

Explore ideas. Compare options. Make room for something new.

Topic

React

From

01:00 PM

Until

06:00 PM

Update sessions

3 sessions between 13:00 and 18:00.

ListCompare

| Session                                    | Time        | Room | Level        | What you'll learn                                     |
|--------------------------------------------|-------------|------|--------------|-------------------------------------------------------|
| React patterns for adaptive views<br>react | 13:00–13:45 | A    | intermediate | Render trusted components from a typed view model.    |
| A faster React screen<br>react             | 14:00–14:45 | A    | advanced     | Measure render work before reaching for optimization. |
| A small React design system<br>react       | 15:00–15:45 | A    | beginner     | Build reusable components with explicit variants.     |

Fictional workshop data. This is not the React Alicante schedule.

Help me choose

 Tool Result 1712 chars

{ "content": [ { "type": "text", "text": "Fictional workshop data\n3 sessions betw...



# How we'll work

Predict the result before changing the code.

Build a small change, then read the diff.

Run it and explain what happened.

Manual coding, pairing and agents all work.

# Intent and presentation

“Show me React sessions this afternoon.”

→ topic: react · startTime: 13:00 · view: list

“Compare them so I can choose.”

→ same filters · view: compare

# The application flow

Model chooses the tool and arguments

Host calls the MCP server

Server returns data and a UI resource reference

Host loads React in a sandbox

React calls tools through the host bridge

# The tool contract

```
show_sessions({  
  topic: "react",  
  startTime: "13:00",  
  endTime: "18:00",  
  view: "compare"  
})
```

Zod validates inputs and structured results.

# Exercise 1 · The session tool

Filter by topic and the complete time window.

Return typed data and useful fallback text.

Try a valid empty result and an invalid input.

30 minutes · 00-start → 01-tool

## Exercise 2 · The React resource

```
_meta: {  
  ui: { resourceUri: "ui://agenda/sessions.html" }  
}
```

useApp → ontolresult → validated React state

35 minutes · 01-tool → 02-react-ui



## Exercise 4 · UI to server

```
const result = await app.callServerTool({  
  name: "show_sessions",  
  arguments: filters  
});
```

```
return sessionResultSchema.parse(  
  result.structuredContent  
);
```

30 minutes · 03-views → final

# Context and conversation

updateModelContext shares the current selection.

sendMessage asks the host to continue.

The local host records these messages.

A conversational host can respond.

# Loading, empty results and errors

Keep the last useful result during a request.

Disable duplicate submissions.

Explain an empty result and how to recover.

Validate data on both sides of the bridge.

Keep useful text when a host cannot render UI.

# Pair challenge

Add a room filter to schema, server and UI.  
Or add a timeline as a new trusted view.

Preserve the other filters. Add one useful test.  
20 minutes · Show the actual result

# Keep building

[github.com/glennreyes/generative-ui-mcp-workshop](https://github.com/glennreyes/generative-ui-mcp-workshop)

Setup · exercises · solutions · slides

Build with Glenn on Discord

[discord.gg/8p3uGHNMu](https://discord.gg/8p3uGHNMu)